

Esercizi di Algoritmi e Strutture Dati

Homework1:

Esercizio 1.1. Notazione asintotica

Per ognuna delle seguenti affermazioni, si dica se essa è sempre vera, mai vera, o a volte vera, per funzioni asintoticamente non-negative. Se la si considera sempre vera o mai vera, si spieghi il perché. Se è a volte vera, si dia un esempio per cui è vera e uno per cui è falsa.

- $f(n) = O(f(n)^2)$
- $f(n) + O(f(n)) = \Theta(f(n))$
- $f(n) = \Omega(g(n))$ e $f(n) = o(g(n))$ — Nota la notazione little-o

Affermazione 1: $f(n) = O(f(n)^2)$

Risposta: **A volte vera**

Per definizione, $f(n) = O(f(n)^2)$ richiede che esistano due costanti $c > 0$ e n_0 tali per cui $f(n) \leq c \cdot f(n)^2$ per ogni $n \geq n_0$.

La verità dell'affermazione dipende dal comportamento di $f(n)$ per n che cresce:

- **Esempio in cui è VERA:** $f(n) = n$. Abbiamo $n = O(n^2)$, che è banalmente vera poiché $n \leq 1 \cdot n^2$ per ogni $n \geq 1$ ($c = 1$, $n_0 = 1$).
- **Esempio in cui è FALSA:** $f(n) = 1/n$. Sostituendo si ottiene $1/n = O(1/n^2)$, ovvero $1/n \leq c / n^2$, che implica $n \leq c$. Questo è impossibile per n che tende all'infinito, poiché c è una costante fissa.

L'affermazione è quindi falsa quando $f(n)$ tende a 0, mentre è vera quando $f(n)$ è limitata inferiormente da una costante positiva (ad esempio $f(n) \geq 1$): in tal caso, dividendo per $f(n)$, si ottiene $1 \leq c \cdot f(n)$, verificata con $c = 1$.

Affermazione 2: $f(n) + O(f(n)) = \Theta(f(n))$

Risposta: **Sempre vera**

Sia $g(n)$ una generica funzione appartenente all'insieme $O(f(n))$. Per definizione di O-grande e sapendo che le funzioni sono asintoticamente non-negative, esistono due costanti positive c e n_0 tali che:

$$0 \leq g(n) \leq c \cdot f(n) \quad \text{per ogni } n \geq n_0$$

Sommando $f(n)$ a ciascun membro della disuguaglianza otteniamo:

$$f(n) \leq f(n) + g(n) \leq f(n) + c \cdot f(n) = (1 + c) \cdot f(n)$$

Possiamo quindi scegliere due nuove costanti positive $c_1 = 1$ e $c_2 = 1 + c$. La relazione precedente diventa:

$$c_1 \cdot f(n) \leq f(n) + g(n) \leq c_2 \cdot f(n) \quad \text{per ogni } n \geq n_0$$

Questo soddisfa esattamente la definizione di $\Theta(f(n))$. Dato che questo vale per qualsiasi funzione $g(n)$ scelta all'interno di $O(f(n))$, l'uguaglianza asintotica $f(n) + O(f(n)) = \Theta(f(n))$ è pienamente verificata.

Affermazione 3: $f(n) = \Omega(g(n))$ e $f(n) = o(g(n))$

Risposta: Mai vera

Dimostrazione per assurdo: Le definizioni di Ω e o sono mutuamente esclusive per funzioni non identicamente nulle, infatti:

- **$f(n) = \Omega(g(n))$** significa che esistono costanti $c > 0$ e n_0 tali che $f(n) \geq c \cdot g(n)$ per ogni $n \geq n_0$ ($f(n)$ cresce ad una velocità $\geq$ a $g(n)$).
- **$f(n) = o(g(n))$** significa che per qualunque costante $\varepsilon > 0$, esiste un n_1 tale che $f(n) < \varepsilon \cdot g(n)$ per ogni $n \geq n_1$ ($f(n)$ cresce in modo strettamente più lento di $g(n)$, ovvero il limite del loro rapporto è 0).

Se provassimo a supporre che entrambe siano vere contemporaneamente, potremmo scegliere il valore $\varepsilon = c / 2$ nella definizione di o-piccolo.

Di conseguenza, per tutti i valori di n sufficientemente grandi ($n \geq \max(n_0, n_1)$), dovrebbero valere contemporaneamente le seguenti disuguaglianze:

$$c \cdot g(n) \leq f(n) < (c / 2) \cdot g(n)$$

Poiché $f(n) = \Omega(g(n))$ con $c > 0$ implica $f(n) \geq c \cdot g(n) > 0$ asintoticamente, $g(n)$ deve essere strettamente positiva per n sufficientemente grande.

Possiamo quindi dividere per $g(n)$ senza alterare il verso delle disuguaglianze, ottenendo: $c \leq f(n)/g(n) < c/2$, il che implica $c < c/2$, ovvero $1 < 1/2$.

Pertanto, l'affermazione è **mai vera**.

Esercizio 1.2. Notazione asintotica e crescita delle funzioni

Per ognuna delle seguenti coppie di funzioni $f(n)$ e $g(n)$, trovare una appropriata costante positiva c tale che $f(n) \leq c \cdot g(n)$ per tutti i valori di $n > 1$.

1. $f(n) = n^2 + n + 1$, $g(n) = 2n^3$
2. $f(n) = n\sqrt{n} + n^2$, $g(n) = n^2$
3. $f(n) = n^2 - n + 1$, $g(n) = n^2/2$

Coppia 1: $f(n) = n^2 + n + 1$, $g(n) = 2n^3$

Vogliamo determinare $c > 0$ tale che $n^2 + n + 1 \leq c \cdot (2n^3)$ per ogni $n > 1$.

Dato che $n > 1$, valgono le seguenti maggiorazioni:

- $n^2 < n^3$
- $n < n^3$
- $1 < n^3$

Sommando i termini otteniamo: $n^2 + n + 1 < n^3 + n^3 + n^3 = 3n^3$.

Imponiamo $3n^3 \leq c \cdot 2n^3 \Rightarrow 3 \leq 2c \Rightarrow c \geq 1.5$.

Scelta della costante: **$c = 1.5$** (oppure qualsiasi valore superiore, ad esempio $c = 2$).

Coppia 2: $f(n) = n\sqrt{n} + n^2$, $g(n) = n^2$

Vogliamo determinare $c > 0$ tale che $n\sqrt{n} + n^2 \leq c \cdot n^2$ per ogni $n > 1$.

Per $n > 1$, si ha che $\sqrt{n} < n$, di conseguenza avrò che $n\sqrt{n} < n^2$.

Possiamo quindi maggiorare $f(n)$ come: $n\sqrt{n} + n^2 < n^2 + n^2 = 2n^2$.

Dal confronto con $c \cdot g(n)$, abbiamo $2n^2 \leq c \cdot n^2 \Rightarrow c \geq 2$.

Pertanto una possibile scelta della costante: **$c = 2$** .

Coppia 3: $f(n) = n^2 - n + 1$, $g(n) = n^2/2$

Vogliamo determinare $c > 0$ tale che $n^2 - n + 1 \leq c \cdot (n^2/2)$ per ogni $n > 1$.

Notiamo che per $n > 1$, il blocco $(-n + 1) < 0$ (infatti $-n + 1 < 0 \Rightarrow n > 1$).

Di conseguenza, $f(n) < n^2$ per ogni $n > 1$.

Imponiamo $n^2 \leq c \cdot (n^2/2) \Rightarrow 1 \leq c/2 \Rightarrow c \geq 2$.

Scelta della costante: **$c = 2$** .

Esercizio 1.3. Complessità

Dimostrare che per qualsiasi costante reale a e per ogni $b > 0$, $(n + a)^b = \Theta(n^b)$

Soluzione:

Richiamo Definizione Theta:

Per dimostrare che $f(n) = \Theta(g(n))$, è necessario trovare due costanti reali positive $c_1, c_2 > 0$ e una soglia intera n_0 , tali che per ogni $n \geq n_0$, risulti vera la seguente doppia disuguaglianza: $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$

Questa condizione esprime che $f(n)$ è "compresa" tra due multipli costanti di $g(n)$: non cresce né troppo velocemente né troppo lentamente rispetto a $g(n)$.

- $f(n) = (n + a)^b \rightarrow$ la funzione da analizzare
- $g(n) = n^b \rightarrow$ la funzione di confronto (termine dominante)

Dobbiamo quindi trovare $c_1, c_2 > 0$ e n_0 : tali che, per ogni $n \geq n_0$:

$$c_1 \cdot (n)^b \leq (n + a)^b \leq c_2 \cdot (n)^b$$

Dimostrazione:

Per analizzare il comportamento asintotico, raccogliamo il termine n all'interno della base della potenza. Per $n > 0$: $(n + a)^b = \left(n \cdot \left(1 + \frac{a}{n}\right)\right)^b = n^b \cdot \left(1 + \frac{a}{n}\right)^b$

Perché questo passaggio è fondamentale?

Abbiamo separato la funzione in due parti ben distinte:

- $(n)^b \rightarrow$ la parte principale, che determina la velocità di crescita quando n diventa grande
- $\left(1 + \frac{a}{n}\right)^b \rightarrow$ un fattore correttivo che dipende da n , ma che si avvicina a 1 quando n cresce

Infatti, quando $n \rightarrow +\infty$: $\frac{a}{n} \rightarrow 0 \Rightarrow \left(1 + \frac{a}{n}\right)^b \rightarrow 1$

Bisogna definire delle costanti esplicite che limitano $\left(1 + \frac{a}{n}\right)^b$ superiormente e inferiormente.

Determinazione di n_0

Scegliamo $n_0 = \lceil 2|a| \rceil$ per limitare il termine correttivo (n_0 è un intero; pertanto quando $2|a| < 1$ si pone $n_0 = 1$, e la condizione resta soddisfatta). In questo modo garantiamo che, per ogni $n \geq n_0$, il valore assoluto di $\frac{a}{n}$ è al massimo $1/2$:

$$\left| \frac{a}{n} \right| = \frac{|a|}{n} \leq \frac{|a|}{n_0} \leq \frac{|a|}{\lceil 2|a| \rceil} \leq \frac{|a|}{2|a|} = \frac{1}{2}$$

Avendo dimostrato che $-\frac{1}{2} \leq \frac{a}{n} \leq \frac{1}{2}$, sommiamo 1, il termine $(1 + \frac{a}{n})$ rimarrà sempre confinato nell'intervallo $[\frac{1}{2}, \frac{3}{2}]$:

$$1 - \frac{1}{2} \leq 1 + \frac{a}{n} \leq 1 + \frac{1}{2}$$

Perchè questa scelta funziona

Gli estremi $\frac{1}{2}, \frac{3}{2}$ sono entrambi strettamente positivi; in questo modo il termine $(1 + \frac{a}{n})$ rimane positivo, consentendo l'elevamento alla potenza $b > 0$

Caso $a = 0$: la condizione è soddisfatta banalmente per $n_0 = 1$, perchè $(n + 0)^b = (n)^b$ annullando la presenza del fattore correttivo.

Elevamento a potenza

Poiché $b > 0$, la funzione x^b è **strettamente crescente** per $x > 0$. Possiamo quindi elevare tutti i membri della disuguaglianza alla potenza b senza invertire il verso:

$$\left(\frac{1}{2}\right)^b \leq \left(1 + \frac{a}{n}\right)^b \leq \left(\frac{3}{2}\right)^b$$

.Per le proprietà delle funzioni crescenti ($x \leq y$, allora $(x)^b \leq (y)^b$), possiamo elevare tutti i membri alla potenza b mantenendo le disuguaglianze. Moltiplicando ora tutti i membri per $(n)^b > 0$, il verso rimane inalterato:

$$\left(\frac{1}{2}\right)^b * (n)^b \leq (n)^b * \left(1 + \frac{a}{n}\right)^b \leq \left(\frac{3}{2}\right)^b * (n)^b$$

Ricordando che $(n)^b * \left(1 + \frac{a}{n}\right)^b = (n + a)^b$ (come mostrato al Passo 1), otteniamo:

$$\left(\frac{1}{2}\right)^b * (n)^b \leq (n + a)^b \leq \left(\frac{3}{2}\right)^b * (n)^b$$

La funzione $(n + a)^b$ è ora "compresa" tra due multipli costanti di $(n)^b$. Questo è esattamente ciò che richiede la definizione di Theta.

Identificando le costanti finali:

$c_1 = \left(\frac{1}{2}\right)^b$	limite inferiore, strettamente > 0
$c_2 = \left(\frac{3}{2}\right)^b$	limite superiore, strettamente > 0
$n_0 = \lceil 2 a \rceil$	soglia a partire dalla quale vale la doppia disug

Risultato finale

Per ogni $n \geq n_0 = \lceil 2|a| \rceil$, la doppia disuguaglianza è soddisfatta:

$$c_1 * (n)^b \leq (n + a)^b \leq c_2 * (n)^b$$

Avendo trovato $c_1, c_2 > 0$ e n_0 espliciti, la condizione della notazione Theta è pienamente soddisfatta.

$$(n + a)^b = \Theta(n)^b$$

Esercizio 1.4. Ricorrenza

Fornire il limite inferiore e superiore per $T(n)$ nella seguente ricorrenza, usando il metodo dell'albero delle ricorrenze ed il teorema dell'esperto se applicabile. Si fornisca il limite più stretto possibile giustificando la risposta.

- $T(n) = 2T(n/3) + n \lg n$
- $T(n) = 3T(n/5) + \lg^2 n$

Ricorrenza 1: $T(n)=2T(n/3)+n \lg n$

1. Metodo dell'Esperto:

La ricorrenza è nella forma $T(n) = aT(n/b) + f(n)$

- **Passo 1: Identificazione dei parametri** $a = 2, b = 3, f(n) = n \lg n$.
- **Passo 2: Calcoliamo il costo asintotico delle foglie** $n^{\log_b a} = n^{\log_3 2} \approx n^{0.631}$.
- **Passo 3: Confronto**

Devo confrontare asintoticamente $f(n)$ con $n^{\log_3 2}$; Osserviamo che $f(n)$ ha un termine lineare in n^1 (moltiplicato per un logaritmo), pertanto $1 > 0.631$, quindi $f(n)$ cresce polinomialmente più veloce rispetto al costo delle foglie.

Quindi, definisco $\varepsilon \approx 0.36$, tale per cui: $f(n) = \Omega(n^{\log_3 2 + \varepsilon})$. Questo ci colloca nel **Caso 3** del Teorema dell'Esperto, purché valga la condizione di regolarità.

- **Passo 4: Verifica della condizione di regolarità**

Devo verificare che esista una costante $c < 1$ tale che, per n sufficientemente grande, sia soddisfatta la seguente disuguaglianza: $a \cdot f(n/b) \leq c \cdot f(n)$

Sostituiamo i valori della nostra ricorrenza: $2 \left(\frac{n}{3} \lg \frac{n}{3} \right) \leq c \cdot n \lg n$

Applicando le proprietà dei logaritmi, riscrivo: $\frac{2}{3} n \lg n - \frac{2}{3} n \lg 3 \leq c \cdot n \lg n$;

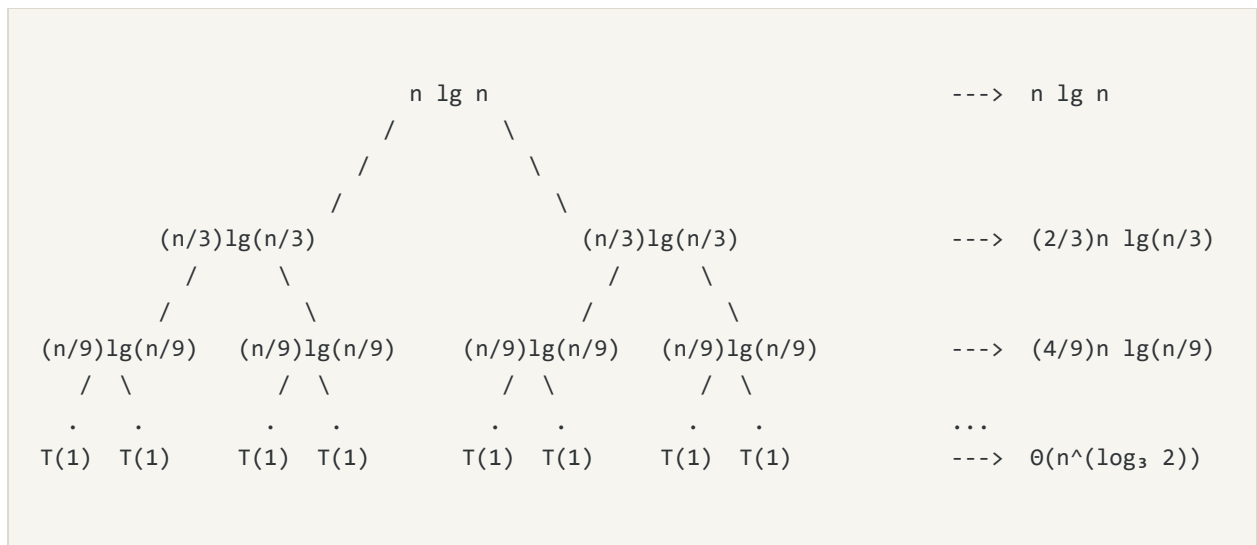
Dividendo per $n \lg n$ si avrà: $\frac{2}{3} - \frac{\frac{2}{3} \lg 3}{\lg n} \leq c$;

Osserviamo che il termine sottratto $\left(\frac{\frac{2}{3} \lg 3}{\lg n} \right)$ è sempre positivo, mentre la

frazione $\frac{2}{3}$ meno qualcosa di positivo sarà sicuramente strettamente minore di $\frac{2}{3}$. La condizione è sempre verificata se scegliamo come costante: $c = \frac{2}{3} < 1$

Conclusione: Il limite stretto asintotico è dominato da $f(n)$ quindi $T(n) = \Theta(n \lg n)$

2. Albero della ricorrenza:



Il costo totale è dato dalla somma dei costi di tutti i livelli interni (da $i = 0$ fino al penultimo livello, ovvero $\log_3 n - 1$ più il costo dell'ultimo livello (le foglie):

$$T(n) = \sum_{i=0}^{\log_3 n - 1} \left[2^i \cdot \frac{n}{3^i} \lg \left(\frac{n}{3^i} \right) \right] + \theta(n^{\log_3 2})$$

possiamo maggiore il termine logaritmico $\lg(n/3^i)$ con $\lg n$, poiché il logaritmo decresce scendendo nell'albero. Questo ci permette di portare fuori i termini indipendenti da i :

$$T(n) \leq \sum_{i=0}^{\log_3 n - 1} \left[\left(\frac{2}{3} \right)^i n \lg n \right] + \theta(n^{\log_3 2}) \leq n \lg n \sum_{i=0}^{\log_3 n - 1} \left(\frac{2}{3} \right)^i + \theta(n^{\log_3 2})$$

maggioriamo la somma finita estendendola all'infinito per usare la formula della serie geometrica decrescente:

$$T(n) \leq n \lg n \sum_{i=0}^{\infty} \left(\frac{2}{3} \right)^i + \theta(n^{\log_3 2}) \leq n \lg n \cdot \left(\frac{1}{1 - 2/3} \right) + \theta(n^{\log_3 2})$$

Quindi: $T(n) \leq 3n \lg n + \theta(n^{0.631})$

Conclusione: Poiché $n \lg n$ cresce asintoticamente molto più velocemente di $n^{0.631}$ il termine lineare-logaritmico "assorbe" il costo delle foglie. Otteniamo quindi: $T(n) = \theta(n \lg n)$

Ricorrenza 2: $T(n) = 3T(n/5) + \lg^2 n$

1. Metodo dell'Esperto:

La ricorrenza è nella forma $T(n) = aT(n/b) + f(n)$

- **Passo 1: Identificazione dei parametri** $a = 3, b = 5, f(n) = \lg^2 n$.
- **Passo 2: Calcoliamo il costo asintotico delle foglie** $n^{\log_b a} = n^{\log_5 3} \approx n^{0.682}$
- **Passo 3: Confronto**

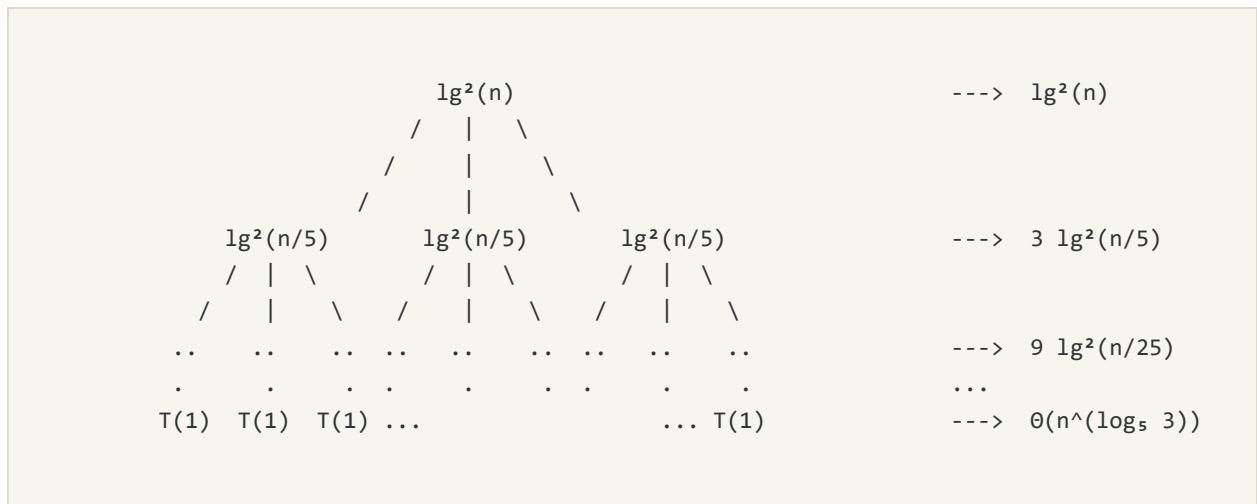
Devo confrontare $f(n)$ con $n^{0.682}$. Per le regole dei limiti asintotici, qualsiasi funzione polinomiale con esponente strettamente maggiore di 0 cresce asintoticamente più velocemente di qualsiasi funzione esprimibile come una potenza del logaritmo. Di conseguenza, il costo delle foglie domina polinomialmente il lavoro locale.

Esiste un $\epsilon > 0$ tale che: $f(n) = O(n^{\log_5 3 - \epsilon})$

Dal confronto emerge, che siamo nel **Caso 1** del Teorema dell'Esperto

Conclusione: Il limite stretto asintotico è interamente dominato dalle foglie, quindi $T(n) = \Theta(n^{\log_5 3})$

2. Albero della ricorrenza:



Impostiamo l'equazione del costo totale sommando il lavoro dei nodi interni e il costo delle foglie relativo all'ultimo livello:

$$T(n) = \sum_{i=0}^{\log_5 n - 1} 3^i \lg^2 \left(\frac{n}{5^i} \right) + \Theta(n^{\log_5 3})$$

In questo caso, non posso maggiorare direttamente $\lg^2 \left(\frac{n}{5^i} \right)$ con $\lg^2 n$ in quanto otterrei un limite superiore troppo largo. Per trovare il limite esatto, **contiamo i livelli dal basso verso l'alto** con un cambio di variabile. Sia j la distanza dal fondo dell'albero, pertanto:

Poniamo $j = \log_5 n - i$, di conseguenza:

$$i = \log_5 n - j$$

$$\text{Il termine } 3^i \text{ diventa } 3^{\log_5 n - j} = \frac{3^{\log_5 n}}{3^j} = \frac{n^{\log_5 3}}{3^j}$$

$$\text{L'argomento del logaritmo } \frac{n}{5^i} \text{ diventa } \frac{n}{5^{\log_5 n - j}} = \frac{n \cdot 5^j}{5^{\log_5 n}} = \frac{n \cdot 5^j}{n} = 5^j$$

Sostituendo tutto nella sommatoria, invertiamo l'indice:

$$T(n) = \sum_{j=1}^{\log_5 n} \left[\frac{n^{\log_5 3}}{3^j} \lg^2(5^j) \right] + \theta(n^{\log_5 3})$$

Applichiamo la proprietà dei logaritmi all'esponente $\lg^2(5^j) = (j \lg 5)^2 = j^2 \lg^2 5$

$$T(n) = \sum_{j=1}^{\log_5 n} \left[n^{\log_5 3} \frac{j^2 \lg^2 5}{3^j} \right] + \theta(n^{\log_5 3})$$

Ora portiamo fuori dalla sommatoria tutti i termini che non dipendono dall'indice j e maggioriamo la serie :

$$T(n) = n^{\log_5 3} \lg^2 5 \sum_{j=1}^{\log_5 n} \frac{j^2}{3^j} + \theta(n^{\log_5 3}) \leq n^{\log_5 3} \lg^2 5 \sum_{j=1}^{\infty} \frac{j^2}{3^j} + \theta(n^{\log_5 3})$$

osserviamo che $\sum_{j=1}^{\infty} \frac{j^2}{3^j}$ è una nota serie convergente: poiché il denominatore 3^j cresce molto più velocemente del numeratore j^2 , la serie converge a una costante reale fissa che chiamiamo K .

Poiché $\lg^2 5$ e K alla fine sono delle semplici costanti numeriche, l'intero costo dei nodi interni si riduce a un multiplo costante di $n^{\log_5 3}$, Pertanto: $T(n) \leq \theta(n^{\log_5 3}) + \theta(n^{\log_5 3})$

Conclusione: Poiché la somma del lavoro interno e del lavoro delle foglie è della stessa grandezza $T(n) = \theta(n^{\log_5 3})$

.

Problema1.

Si implementi un algoritmo di ordinamento che sfrutta l'inserimento e la visita in un albero binario di ricerca. Dato un vettore di n numeri interi in input, l'algoritmo procede prima ad inserire i numeri in un albero binario di ricerca (usando ripetutamente TREE-INSERT per inserire i numeri uno alla volta), e poi stampa i numeri in ordine con un attraversamento in ordine simmetrico dell'albero. Si analizzi la complessità nel caso peggiore e nel caso migliore di per questo algoritmo di ordinamento.

1. File di Intestazione: funzioni.h

```
#ifndef FUNZIONI_H
#define FUNZIONI_H
#define MAX 100
/*Definizione Nodo*/
typedef struct Nodo{
    /* data */
    int key;
    struct Nodo * left;
    struct Nodo * right;
}Nodo;
/*Prototipi*/
/*Metodi per gestire i casi di test*/
void CreazioneAlbero(int V[],int dim);
void StampaVettore(int V[],int dim);
/*Metodi per gestire l'albero*/
Nodo *CreateNodo(int value);
Nodo * InsertNodo(Nodo *T, Nodo *z);
void InorderTreeWalk(Nodo *x);
#endif
```

2. File delle Funzioni: funzioni.c

```
#include <stdio.h>
#include <stdlib.h>
#include "funzioni.h" // Include il file header

void CreazioneAlbero(int V[],int dim){
    //1.Costruzione dell'albero binario di ricerca
    Nodo *root = NULL;
    for (int i = 0; i < dim; i++) {
        Nodo *nuovoNodo = CreateNodo(V[i]);
        if (nuovoNodo != NULL) {
            root = InsertNodo(root, nuovoNodo);
        } else {

```

```

        printf("Errore:Impossibile allocare memoria per il valore %d\n", V[i]);
    }
}
//2.Stampa
InorderTreeWalk(root);
printf("\n\n");
}
/*Stampa caso di test*/
void StampaVettore(int V[], int dim) {
    for (int i = 0; i < dim; i++)printf("%d ", V[i]);
    printf("\n");
}
/*Creazione del Nodo corrente*/
Nodo *CreateNodo(int value){
    //1.Istanzio il nodo
    Nodo * newNode = malloc(sizeof(*newNode));
    //2.Inizializzo il nodo
    if (newNode==NULL) return NULL;
    newNode->key=value;
    newNode->left=NULL;
    newNode->right=NULL;
    //3.Ritorna il nodo creato
    return newNode;
}
/*Inserimento nodo z all'interno dell'albero T*/
Nodo* InsertNodo(Nodo *T, Nodo *z){
    //Passo1:Definisco un nodo d'appoggio
    Nodo * y=NULL;
    Nodo *x=T;
    //Passo2:Inizio la ricerca della posizione spostandomi all'interno
    dell'albero
    while (x!=NULL){
        //salvo la pos corrente di x,tenendo traccia dell'ultimo nodo visitato
        y=x;
        if (z->key<x->key)x=x->left;//Vado a sx
        else x=x->right;//vado a dx
    }
    //Passo3:Posiziono il nodo z
    if(y==NULL)
        T=z;//z è la radice
    else if(z->key<y->key)
        y->left=z; //z è figlio sx
    else
        y->right=z;//z è figlio dx
    //Passo4:Ritorna l'albero aggiornato
    return T;
}

```

```

}
/*Attraverso questo metodoandrò ad effettuare la stampa degli n nodi in modo
crescente. Poiché vado a visitare ogni nodo una voltaavrò una complessità
Theta(n)
*/
void InorderTreeWalk(Nodo *x){
    if (x!=NULL){
        InorderTreeWalk(x->left);//scendo a sx
        printf("%d ",x->key);
        InorderTreeWalk(x->right);//scendo a dx
    }
}

```

3. Programma Principale: main.c

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include "funzioni.h" // Include le funzioni definite in funzioni.c
int main() {
    //1.Definisco i casi di test
    printf("SAMPLE INPUT:");
    /* TEST 1: Caso Medio (Array disordinato)
    L'albero risultante sarà moderatamente bilanciato.
    In questo scenario, la complessità temporale per la costruzione dell'albero
    è in media  $O(n \log n)$ .*/
    int array1[] = {5, 3, 7, 2, 4, 8};
    int dim1 = sizeof(array1) / sizeof(array1[0]);
    printf("\n Caso di Test1:");
    StampaVettore(array1,dim1);
    /* TEST 2: Caso Peggior 1 (Array già ordinato in modo crescente)
    Ogni nuovo nodo diventa il figlio destro del nodo precedente.
    L'albero degenera in una lista concatenata verso destra.
    La complessità per l'inserimento sale a  $O(n^2)$ .*/
    int array2[] = {1, 2, 3, 4, 5, 6, 7};
    int dim2 = sizeof(array2) / sizeof(array2[0]);
    printf("\n Caso di Test2:");
    StampaVettore(array2,dim2);
    /*TEST 3: Caso Peggior 2 (Array ordinato in modo decrescente)
    Ogni nuovo nodo diventa il figlio sinistro del nodo precedente.
    L'albero degenera in una lista concatenata verso sinistra.
    Anche qui, la complessità per l'inserimento sale a  $O(n^2)$ .*/
}

```

```

    int array3[] = {9, 8, 7, 6, 5, 4, 3};
    int dim3 = sizeof(array3) / sizeof(array3[0]);
    printf("\n Caso di Test3:");
    StampaVettore(array3,dim3);
    //Stampa casi di test
    printf("SAMPLE OUTPUT:");
    printf("\n Output1:");
    CreazioneAlbero(array1, dim1);
    printf("\n Output2:");
    CreazioneAlbero(array2, dim2);
    printf("\n Output3:");
    CreazioneAlbero(array3, dim3);
    system("pause");
    return 0;
}

```

4. Output Atteso

```

SAMPLE INPUT:
Caso di Test1:5 3 7 2 4 8

Caso di Test2:1 2 3 4 5 6 7

Caso di Test3:9 8 7 6 5 4 3
SAMPLE OUTPUT:
Output1:2 3 4 5 7 8

Output2:1 2 3 4 5 6 7

Output3:3 4 5 6 7 8 9

Premere un tasto per continuare . . .

```

5. Analisi Complessità:

La complessità complessiva dipende interamente dalla forma che assume l'albero durante la fase di costruzione, poiché i nodi vengono inseriti rispettando solo il vincolo di ricerca e non un criterio di bilanciamento.

Caso Migliore: $\Theta(n \log n)$	Caso Peggior: $\Theta(n^2)$
Questo scenario si verifica nel caso migliore (input con disposizione ottimale) e rappresenta anche il comportamento atteso nel caso medio (input generato casualmente). In entrambi i casi la complessità è $\Theta(n \log n)$: l'albero mantiene altezza $\Theta(\log n)$, gli inserimenti costano $\Theta(n \log n)$ in totale, e l' InorderTreeWalk aggiunge $\Theta(n)$	Se l'input è già ordinato (crescente o decrescente), l'albero degenera in una lista concatenata. L'altezza diventa $\Theta(n)$, e l'i-esimo inserimento percorre esattamente $i-1$ nodi; quindi, il costo totale degli inserimenti è $\Theta(n^2)$. La visita InorderTreeWalk mantiene $\Theta(n)$. Totale: $\Theta(n^2)$.

Problema2.

Problema 2. Si implementi un algoritmo che, a partire da un vettore di n numeri interi in input, costruisce un heap chiamando ripetutamente la procedura MAX-HEAP-INSERT (vedi slide su heapsort) per inserire gli elementi nell'heap. L'algoritmo di costruzione ha il seguente pseudocodice:

```
BUILD-MAX-HEAP_v2(A)
  A.heap_size = 1
  for (i=2) to A.length
    MAX-HEAP-INSERT(A, A[i])
```

Si confronti la BUILD-MAX-HEAP vista a lezione con la BUILD-MAX-HEAP_v2, in particolare: le due procedure creano sempre lo stesso heap se vengono eseguite con lo stesso array di input? Dimostrare che lo fanno o fornite un controesempio. Dimostrare che, nel caso peggiore, BUILD-MAX-HEAP_v2 richiede un tempo $\Theta(n \log n)$ per costruire un heap di n elementi.

1. Analisi:

Quesito 1: Le due procedure creano sempre lo stesso heap se vengono eseguite con lo stesso array di input? Dimostrare che lo fanno o fornite un controesempio.

No, le due procedure **non** creano sempre lo stesso heap. Anche se entrambe producono un Max-Heap valido (dove ogni nodo padre è maggiore o uguale ai suoi figli), la disposizione interna dei nodi può variare a causa dell'approccio strutturalmente opposto:

- La **BUILD-MAX-HEAP classica** procede dal basso verso l'alto (*bottom-up*), facendo "scendere" i valori più piccoli tramite la procedura MAX-HEAPIFY.
- La **BUILD-MAX-HEAP_v2** procede dall'alto verso il basso (*top-down*), inserendo i nuovi elementi come foglie e facendoli "salire" se sono maggiori dei padri tramite MAX-HEAP-INSERT.

Poiché le due procedure non generano sempre lo stesso heap: forniamo un controesempio.

Controesempio:

Per dimostrare questa differenza, prendiamo in esame un piccolo array di 3 elementi ordinato in modo crescente: $A = [1, 2, 3]$

Esecuzione 1: Algoritmo Classico (BUILD-MAX-HEAP)

L'algoritmo interpreta l'array come un albero in cui la radice è 1, il figlio sinistro è 2 e il figlio destro è 3. Il ciclo parte dall'ultimo nodo non foglia (l'indice 1, ovvero la radice).

1. Viene chiamata $\text{MAX-HEAPIFY}(A, 1)$.
2. Il valore del padre (1) viene confrontato con i figli (2 e 3).
3. Il figlio maggiore è 3. Avviene uno scambio tra il padre e il figlio destro.
4. **Vettore finale V1:** $[3, 2, 1]$

Esecuzione 2: Variante con Inserimento (BUILD-MAX-HEAP_v2)

L'algoritmo inizia con un heap di dimensione 1 e inserisce i successivi iterativamente.

1. Pongo $\text{HeapSize} = 1$.
2. **Inserimento di 2:** L'elemento viene aggiunto in coda: $[1, 2]$. Il 2 è maggiore del padre (1). La procedura MAX-HEAP-INSERT li scambia. L'heap diventa $[2, 1]$.
3. **Inserimento di 3:** L'elemento viene aggiunto in coda: $[2, 1, 3]$. Il 3 (figlio destro) è maggiore del padre (2). Viene effettuato uno scambio. L'heap diventa $[3, 1, 2]$.
4. **Vettore finale V2:** $[3, 1, 2]$.

Conclusione: Partendo dall'array di input $[1, 2, 3]$, otteniamo $[3, 2, 1]$ con la procedura classica e $[3, 1, 2]$ con la V2. Gli array differiscono, confermando la validità del controesempio.

Quesito 2: Dimostrare che, nel caso peggiore, BUILD-MAX-HEAP_v2 richiede un tempo $\Theta(n \log n)$ per costruire un heap di n elementi.

Analisi MAX-HEAP-INSERT

L'algoritmo si basa sulle chiamate ripetute alla procedura MAX-HEAP-INSERT . Questa funzione inserisce l'elemento in fondo all'heap e lo fa risalire finché la proprietà dell'heap è ripristinata. Il costo è proporzionale all'altezza dell'heap: inserendo l' i -esimo elemento, l'heap ha dimensione $i-1$ e altezza $\lfloor \log_2(i-1) \rfloor$. Pertanto, il costo nel caso peggiore è: $O(\log i)$;

Definizione del Caso Peggiore

Il caso peggiore per la procedura di costruzione BUILD-MAX-HEAP_v2 si verifica quando l'array di input è **strettamente crescente** (es. $[1, 2, 3, \dots, n]$). In questo scenario, ogni i -esimo elemento inserito sarà sempre più grande di tutti i precedenti. Pertanto, l'algoritmo dovrà fargli percorrere sempre l'intera altezza dell'albero, dalle foglie fino a diventare la nuova radice.

Analisi della complessità temporale

Per calcolare la ricorrenza nel caso peggiore, dobbiamo sommare i costi di tutti gli inserimenti da $i = 2$ fino a n : (NOTA: Si parte da $i = 2$ perché il primo elemento, inserito in $A[1]$, costituisce già una Max-Heap di un solo nodo e non richiede operazioni di risalita.)

$$T(n) = \sum_{i=2}^n O(\log_2 i)$$

Per dimostrare che $T(n) = \Theta(n \log n)$ analizziamo separatamente il limite superiore (O) e il limite inferiore (Ω):

1. **Limite Superiore (O):** Possiamo maggiorare la sommatoria sostituendo ogni termine con il valore massimo possibile nell'intervallo, ovvero $\log_2 n$:

$$T(n) = \sum_{i=2}^n \log_2 i \leq \sum_{i=2}^n \log_2 n \leq n \log_2 n \Rightarrow T(n) = O(n \log n)$$

2. **Limite Inferiore (Ω):** Possiamo minorare la sommatoria considerando solo la seconda metà dei termini (da $\lfloor n/2 \rfloor$ a n), dove ogni termine è sicuramente maggiore o uguale a $\log_2(n/2)$:

$$T(n) = \sum_{i=2}^n \log_2 i \geq \sum_{i=\lfloor n/2 \rfloor}^n \log_2 i \geq \sum_{i=\lfloor n/2 \rfloor}^n \log_2 \left(\frac{n}{2}\right)$$

Poiché la seconda metà contiene circa $\frac{n}{2}$ termini, otteniamo:

$$T(n) \geq \frac{n}{2} \cdot \log_2 \left(\frac{n}{2}\right) = \frac{n}{2} (\log_2 n - \log_2 2) = \frac{1}{2} n \log_2 n - \frac{n}{2}$$

Per valori di n asintoticamente grandi, il termine lineare $\frac{n}{2}$ viene ignorato rispetto al termine dominante, di conseguenza: $T(n) = \Omega(n \log n)$

Conclusione: Avendo dimostrato che il tempo di esecuzione nel caso peggiore è contemporaneamente $O(n \log n)$ e $\Omega(n \log n)$, per definizione stessa di limite asintotico stretto, concludiamo che: $T(n) = \Theta(n \log n)$

2. File di Intestazione: funzioni.h

```
#ifndef FUNZIONI_H
#define FUNZIONI_H
#define MAX 100
/*Definisco la mia struttura Heap*/
typedef struct{
    int HeapSize;
    int lenght;
    int v[MAX];
} Struttura_Heap;

/* Metodi di utilità */
int Parent(int i);
int Left(int i);
int Right(int i);
void Swap(int *a, int *b);
void StampaVettore(int arr[], int dim);
void EseguiTest(int arr[], int dim, const char* nome_test);
/* Metodi Heap classici (V1) */
void MaxHeapfy(Struttura_Heap *A, int i);
void BuildMaxHeap(Struttura_Heap *A);
/* Metodi Heap V2 (con inserimento) */
void MaxHeapInsert(Struttura_Heap *A, int i);
void BuildMaxHeapV2(Struttura_Heap *A);
#endif
```

3. File delle Funzioni: funzioni.c

```
#include <stdio.h>
#include <stdlib.h>
#include "funzioni.h" // Include il file header
/* --- METODI DI UTILITÀ --- */
void EseguiTest(int arr[], int dim, const char* nome_test) {
    //Passo0:Inizializzo la mia struttura Heap V1 e V2
    Struttura_Heap H_V1, H_V2;
    H_V1.lenght = dim;
    H_V2.lenght = dim;
    //Passo1:Popoliamo entrambe le strutture
    for (int i = 0; i < dim; i++) {
        H_V1.v[i] = arr[i];
        H_V2.v[i] = arr[i];
    }
    // Passo 2: Costruiamo SOLO gli Heap, non applico l'intero HeapSort.
    // Come richiesto dalla traccia, vogliamo verificare la differenza tra le
    strutture generate dai due metodi di costruzione.
    BuildMaxHeap(&H_V1);
    BuildMaxHeapV2(&H_V2);
    printf("\nRisultati %s:\n", nome_test);
    printf("    -> Output V1 (Classica): ");
    StampaVettore(H_V1.v, H_V1.lenght);
    printf("\n    -> Output V2 (Insert): ");
    StampaVettore(H_V2.v, H_V2.lenght);
    printf("\n");
}
void Swap(int *a, int *b) {
    int tmp = *a;
    *a = *b;
    *b = tmp;
}
/*Metodi per trovare il padre e figlio sx e dx dato un indice i*/
int Parent(int i){return (i-1)/2;}
int Left(int i){return 2*i+1;}
int Right(int i){return 2*i+2;}
void StampaVettore(int arr[], int dim) {
    printf("[ ");
    for (int i = 0; i < dim; i++) printf("%d ", arr[i]);
    printf("]");
}
```

```

/* --- VERSIONE 1: BUILD-MAX-HEAP CLASSICA --- */
void MaxHeapfy(Struttura_Heap *A, int i) {
    int largest = i; // Inizializza largest come radice
    int l = Left(i);
    int r = Right(i);
    //Controllo1: Se il figlio sinistro è nel range ed è maggiore del padre
    if (l < A->HeapSize && A->v[l] > A->v[largest]) largest = l;
    //Controllo2: Se il figlio destro è nel range ed è maggiore del largest
    attuale
    if (r < A->HeapSize && A->v[r] > A->v[largest])largest = r;
    // Se il più grande non è la radice, scambia e continua a scendere
    if (largest != i) {
        Swap(&A->v[i], &A->v[largest]);
        MaxHeapfy(A, largest);
    }
}

void BuildMaxHeap(Struttura_Heap *A) {
    A->HeapSize = A->lenght;
    // Partiamo dall'ultimo nodo che ha almeno un figlio
    int pos = (A->lenght / 2) - 1;
    for (int i = pos; i >= 0; i--) MaxHeapfy(A, i);
}

/* --- VERSIONE 2: BUILD-MAX-HEAP CON INSERIMENTO --- */
void MaxHeapInsert(Struttura_Heap *A, int i) {
    //Passo1:Inizio la risalita dell'albero
    while (i>0 && A->v[i]>A->v[Parent(i)]){
        // Scambia il nodo con suo padre
        Swap(&A->v[i],&A->v[Parent(i)]);
        // Aggiorna l'indice per continuare a salire
        i=Parent(i);
    }
}

void BuildMaxHeapV2(Struttura_Heap *A) {
    //Passo1:Inizialmente l'heap contiene solo il primo elemento
    A->HeapSize = 1;
    //Passo2:Inseriamo un elemento alla volta, partendo dal secondo
    for (int i = 1; i < A->lenght; i++) {
        A->HeapSize++; // Aumentiamo la dimensione dell'heap
        MaxHeapInsert(A, i); // Facciamo salire il nuovo elemento se necessario
    }
}

```

4. Programma Principale: main.c

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include "funzioni.h" // Include le funzioni definite in funzioni.c

int main() {
    //Passo1:Definisco i casi di test
    printf("SAMPLE INPUT:");
    /* TEST 1: Caso Medio (Array disordinato)
    Un array casuale per verificare il corretto funzionamento di base. */
    int array1[] = {5, 3, 7, 2, 4, 8};
    int dim1 = sizeof(array1) / sizeof(array1[0]);
    printf("\nCaso di Test 1:");
    StampaVettore(array1, dim1);
    /* TEST 2: Caso Peggioro(Array già ordinato in modo crescente)
    Questo è il caso ideale per dimostrare che le due procedure generano heap
    visibilmente diversi. */
    int array2[] = {1, 2, 3, 4, 5, 6, 7};
    int dim2 = sizeof(array2) / sizeof(array2[0]);
    printf("\nCaso di Test 2:");
    StampaVettore(array2, dim2);
    /* TEST 3: Caso Migliore per V2 (Array già ordinato in modo decrescente)
    In questo caso gli elementi inseriti come foglie sono già più piccoli dei
    padri,quindi non ci sarà bisogno di farli risalire. */
    int array3[] = {9, 8, 7, 6, 5, 4, 3};
    int dim3 = sizeof(array3) / sizeof(array3[0]);
    printf("\nCaso di Test 3: ");
    StampaVettore(array3, dim3);
    printf("\n\n");
    //Passo2:Parto con l'ordinamento
    printf("SAMPLE OUTPUT:");
    // Eseguiamo i test
    EseguiTest(array1, dim1, "Test 1 (Disordinato)");
    EseguiTest(array2, dim2, "Test 2 (Controesempio)");
    EseguiTest(array3, dim3, "Test 3 (Decrescente)");
    system("pause");
    return 0;
}

/*
Complessità:
BUILD-MAX-HEAP:Theta(n)
BUILD-MAX-HEAP_v2:Theta(n*logn)
*/
```

5. Output Atteso:

```
SAMPLE INPUT:
Caso di Test 1:[ 5 3 7 2 4 8 ]
Caso di Test 2:[ 1 2 3 4 5 6 7 ]
Caso di Test 3: [ 9 8 7 6 5 4 3 ]

SAMPLE OUTPUT:
Risultati Test 1 (Disordinato):
-> Output V1 (Classica): [ 8 4 7 2 3 5 ]
-> Output V2 (Insert): [ 8 4 7 2 3 5 ]

Risultati Test 2 (Controesempio):
-> Output V1 (Classica): [ 7 5 6 4 2 1 3 ]
-> Output V2 (Insert): [ 7 4 6 1 3 2 5 ]

Risultati Test 3 (Decrescente):
-> Output V1 (Classica): [ 9 8 7 6 5 4 3 ]
-> Output V2 (Insert): [ 9 8 7 6 5 4 3 ]
Premere un tasto per continuare . . .
```

Problema3.

Sia $A[1 \dots n]$ un array di n elementi distinti "quasi ordinato", ovvero in cui ogni elemento dell'array si trova entro k slot dalla sua posizione corretta. Definiamo una "inversione": se $i < j$ e $A[i] > A[j]$, allora la coppia (i, j) è detta inversione di A . Si implementi un algoritmo che ordina il vettore A in tempo $(n \lg k)$. Suggerimento: Si usi un Heap

1. File di Intestazione: funzioni.h

```
#ifndef FUNZIONI_H
#define FUNZIONI_H
#define MAX 100
/*Definisco la mia struttura Heap*/
typedef struct{
    int HeapSize;
    int lenght;
    int v[MAX];
} Struttura_Heap;
/* Metodi di utilità */
void Swap(int *a, int *b);
void StampaVettore(int arr[], int dim);
void EseguiTest(Struttura_Heap *A, int V[], int dim, int k);
/*Metodi per implementare MinHeap*/
int Parent(int i);
int Left(int i);
int Right(int i);
void MinHeapfy(Struttura_Heap *A, int i);
void BuildMinHeap(Struttura_Heap *A);
#endif
```

2. File delle Funzioni: funzioni.c

```
#include <stdio.h>
#include <stdlib.h>
#include "funzioni.h" // Include il file header

/*Metodi di utilità*/
void StampaVettore(int arr[], int dim) {
    for (int i = 0; i < dim; i++) {
        printf("%d ", arr[i]);
    }
}

void Swap(int *a, int *b) {
    int tmp = *a;
    *a = *b;
    *b = tmp;
}

/*Metodi per implementare MinHeap*/

/*Metodi per trovare il padre e figlio sx e dx dato un indice i*/
int Parent(int i){return (i-1)/2;}
int Left(int i){return 2*i+1;}
int Right(int i){return 2*i+2;}

void MinHeapfy(Struttura_Heap *A, int i) {
    int smallest = i; // Inizializza smallest come radice
    int l = Left(i);
    int r = Right(i);
    //Controllo1: Se il figlio sinistro è nel range ed è minore del padre
    if (l < A->HeapSize && A->v[l] < A->v[smallest]) smallest = l;
    //Controllo2: Se il figlio destro è nel range ed è minore del largest
    //attuale
    if (r < A->HeapSize && A->v[r] < A->v[smallest])smallest = r;
    // Se il più piccolo non è la radice, scambia e continua a scendere
    if (smallest != i) {
        Swap(&A->v[i], &A->v[smallest]);
        MinHeapfy(A, smallest);
    }
}

void BuildMinHeap(Struttura_Heap *A) {
    A->HeapSize = A->lenght;
    // Partiamo dall'ultimo nodo che ha almeno un figlio
    int pos = (A->lenght / 2) - 1;
    for (int i = pos; i >= 0; i--) MinHeapfy(A, i);
}
```

```

void EseguiTest(Struttura_Heap *A,int V[],int dim,int k){
    //Passo1:Definizione variabili
    int Risultato[MAX]; // Vettore di appoggio per il risultato ordinato
    int index = 0; // Indice per riempire il vettore Risultato
    //Passo2:Imposto la dimensione iniziale dell'Heap
    A->HeapSize=(k+1<dim)?k+1 : dim;
    //Aggiorno la dimensione di A
    A->lenght = A->HeapSize;
    //Passo3: Inizializzazione e costruzione del MinHeap
    for (int i = 0; i < A->HeapSize; i++)A->v[i] = V[i];
    BuildMinHeap(A);
    //Passo 4: Scorro il resto dell'array, estraggo il minimo e inserisco il
    nuovo elemento ripristinando la proprietà di MinHeap
    for (int i = A->HeapSize; i < dim; i++){
        Risultato[index++]=A->v[0];//Salvo il minimo
        A->v[0]=V[i];//Inserisco il nuovo elemento nella radice
        MinHeapfy(A,0);//Ripristino l'Heap partendo sempre dalla radice
    }
    // Passo 5: Svuoto l'Heap estraendo i minimi rimanenti
    while (A->HeapSize>0){
        Risultato[index++]=A->v[0];
        //L'ultimo elemento è all'indice HeapSize - 1
        A->v[0] = A->v[A->HeapSize-1]; // Sposto l'ultimo elemento in radice
        A->HeapSize--;
        //Sec ci sono ancora elementi ripristino la proprietà di MinHeap
        if (A->HeapSize>0)MinHeapfy(A,0);
    }
    //Passo 6:Stampo il vettore ordinato
    StampaVettore(Risultato,index);
}

```

3. Programma Principale: main.c

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include "funzioni.h" // Include le funzioni definite in funzioni.c
int main() {
    // Inizializzo la struttura Heap che passeremo alla funzione
    Struttura_Heap heap;
    // Passo 1: Definisco i casi di test
    printf("SAMPLE INPUT:\n");
    /* TEST 1: Caso Base (Array k-ordinato)
    Un array disordinato ma in cui ogni elemento dista al massimo k=2 dalla sua
    posizione finale. */
    int array1[] = {2, 3, 1, 5, 6, 4};
    int dim1 = sizeof(array1) / sizeof(array1[0]);
    int k1 = 2;
    printf("\nCaso di Test 1 (k=%d): ", k1);
    StampaVettore(array1, dim1);

    /* TEST 2: Caso Migliore (Array gia' ordinato)
    L'array e' gia' ordinato, ogni elemento dista 0 posizioni dalla sua
    collocazione finale, usiamo un k piccolo. */
    int array2[] = {1, 2, 3, 4, 5, 6, 7};
    int dim2 = sizeof(array2) / sizeof(array2[0]);
    int k2 = 1;
    printf("\nCaso di Test 2 (k=%d): ", k2);
    StampaVettore(array2, dim2);

    /* TEST 3: Caso Peggior (Array ordinato in modo decrescente)
    In questo caso, gli elementi sono all'opposto della loro posizione corretta.
    Affinché l'algoritmo funzioni, k deve per forza coprire tutto l'array (k =
    dim - 1). */
    int array3[] = {9, 8, 7, 6, 5, 4, 3};
    int dim3 = sizeof(array3) / sizeof(array3[0]);
    int k3 = dim3 - 1; // k deve essere 6 in questo caso
    printf("\nCaso di Test 3 (k=%d): ", k3);
    StampaVettore(array3, dim3);
    printf("\n\n");
    // Passo 2: Parto con l'ordinamento
    printf("SAMPLE OUTPUT:\n");
    printf("\nTest 1: \n");
    EseguiTest(&heap, array1, dim1, k1);
    printf("\nTest 2: \n");
    EseguiTest(&heap, array2, dim2, k2);
    printf("\nTest 3: \n");
    EseguiTest(&heap, array3, dim3, k3);
}
```

```

    printf("\n");
    system("pause");
    return 0;
}
/*
    Complessità temporale:  $O(n \log k)$ , dove:
    n è il numero totale di elementi nell'array
    k è la distanza massima di un elemento dalla sua posizione ordinata finale
    (finestra).
*/

```

4. Output Atteso:

```

SAMPLE INPUT:

Caso di Test 1 (k=2): 2 3 1 5 6 4
Caso di Test 2 (k=1): 1 2 3 4 5 6 7
Caso di Test 3 (k=6): 9 8 7 6 5 4 3

SAMPLE OUTPUT:

Test 1:
1 2 3 4 5 6
Test 2:
1 2 3 4 5 6 7
Test 3:
3 4 5 6 7 8 9
Premere un tasto per continuare . . .

```

Homework2:

Problema1.

Si consideri una matrice di 0 ed 1, in cui “1” indica “posizione occupata” e “0” indica “posizione libera”. Si scriva un algoritmo per determinare la sottomatrice massima libera (ossia che contiene tutti 0). L’algoritmo deve riportare il numero di 0 di tale sottomatrice.

Per risolvere il seguente problema applico la **Programmazione Dinamica**: la matrice binaria viene elaborata riga per riga per costruire dinamicamente un istogramma delle altezze, su cui poi viene calcolata l'area del rettangolo massimo, che sarà il nostro risultato.

1. File di Intestazione: funzioni.h

```
#ifndef FUNZIONI_H
#define FUNZIONI_H
#define DIM_N 100
#define DIM_M 100
/*Var Globale*/
extern int DP[DIM_N][DIM_M];
/*Metodi*/
void InitDP(int DP[DIM_N][DIM_M], int N, int M);
int CalcolaDP(int DP[DIM_N][DIM_M], int Matrice[DIM_N][DIM_M], int N, int M);
int CalcolaMassimaArea(int H[DIM_M], int M);
int max2(int a, int b);
#endif
```

2. File delle Funzioni: funzioni.c

```
#include <stdio.h>
#include <stdlib.h>
#include "funzioni.h" // Include il file header
/*Inizializzazione DP*/
void InitDP(int DP[DIM_N][DIM_M], int N, int M){
    for (int i = 0; i < N; i++){
        for (int j = 0; j < M; j++)DP[i][j]=0;
    }
}
int max2(int a, int b){return(a>b)?a:b;}
/*Costruisce la matrice DP (istogramma) riga per riga*/
int CalcolaDP(int DP[DIM_N][DIM_M], int Matrice[DIM_N][DIM_M], int N, int M){
    int MaxArea=0;
    // 1. Costruisco la matrice DP: ogni riga rappresenterà le altezze (H)
    //dell'istogramma
    for (int i = 0; i < N; i++){//Per ogni riga i-esima
        for (int j = 0; j < M; j++){//Per ogni colonna j-esima
            if (Matrice[i][j]==1){
                DP[i][j]=0;// H si azzera se trova un ostacolo
            }else{
                if (i==0){
                    DP[i][j]=1;//Caso base prima riga: altezza iniziale
                }else{
                    DP[i][j]=DP[i-1][j]+1; // Passo induttivo: si somma alla
                    //cella superiore
                }
            }
        }
        //2. Per ogni riga calcoliamo l'area massima dell'istogramma appena
        //costruito
        int AreaRigaX=CalcolaMassimaArea(DP[i],M);
        MaxArea=max2(MaxArea,AreaRigaX);
    }
    //3.Ritorna la soluzione globale ottima
    return MaxArea;
}
/* Calcola l'area del rettangolo massimo in un istogramma */
int CalcolaMassimaArea(int H[DIM_M], int M){
    int MaxArea=0;
```

```

    for (int i = 0; i < M; i++){// Per ogni barra i dell'istogramma
        int h_corrente=H[i];
        if(h_corrente==0)continue;
        int larghezza=1;
        int pos=i;
        // 1. Calcolo la larghezza del rettangolo massimo avente altezza
// h_corrente
        // A. Estensione a sinistra: mi sposto verso sx finché l'altezza
        // adiacente
        // è maggiore o uguale a h_corrente (fermandomi se è strettamente minore)
        for(int j=pos-1;j>=0 && h_corrente<=H[j];j--)larghezza++;
        // B. Estensione a destra: mi sposto verso dx finché l'altezza adiacente
        // è maggiore o uguale a h_corrente (fermandomi se è strettamente
//minore)
        for(int j=pos+1;j<M && h_corrente<=H[j];j++)larghezza++;
        // 2. Calcolo l'area per l'altezza corrente e aggiorno il massimo se
//necessario
        int areaC=larghezza*h_corrente;
        if(areaC>MaxArea)MaxArea=areaC;
    }
    return MaxArea;
}

```

3. Programma Principale: main.c

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include "funzioni.h" // Include le funzioni definite in funzioni.c

int main() {
    printf("SAMPLE INPUT:\n\n");

    // Definisco i casi di test
    int CasiTest = 2;
    int inputN[2] = {6, 6}; // riga iesima associata ad ogni caso di test
    int inputM[2] = {7, 7}; // colonna iesima associata ad ogni caso di test
    int inputMatrici[2][6][7] = {
        // --- Primo caso di test ---
        {
            {0, 1, 1, 0, 1, 1, 0},
            {0, 0, 0, 0, 0, 1, 0},
            {1, 0, 0, 0, 0, 0, 1},
            {0, 1, 0, 0, 0, 0, 1},

```

```

        {1, 1, 0, 0, 0, 1, 0},
        {1, 1, 0, 1, 1, 0, 0}
    },

// --- Secondo caso di test ---
{
    {0, 1, 1, 0, 1, 1, 0},
    {0, 0, 0, 0, 0, 1, 0},
    {1, 1, 0, 0, 0, 0, 1},
    {0, 1, 0, 1, 0, 0, 1},
    {1, 1, 0, 0, 0, 1, 0},
    {1, 1, 0, 1, 1, 0, 0}
}
};
int N = 0, M = 0; // dimensioni matrice N*M
int Matrice[DIM_N][DIM_M];
int index = 0;
int Risultato[DIM_N]; // Array per salvare i risultati
// Definisco la mia DP
int DP[DIM_N][DIM_M];
while (index < CasiTest) {
    // Fase 1: Inserimento
    N = inputN[index];
    M = inputM[index];
    // Copio i parametri nella matrice di appoggio e li stampo
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < M; j++) {
            Matrice[i][j] = inputMatrici[index][i][j];
            printf("%d ", Matrice[i][j]); // Stampa l'elemento appena
//copiato
        }
        printf("\n");
    }
    printf("\n");
    // Fase 2: Calcolo DP
    InitDP(DP, N, M);
    Risultato[index] = CalcolaDP(DP, Matrice, N, M);
    // Fase 3: Passo al prossimo caso di test
    index++;
}
// Fase 4: Stampa Risultati
printf("SAMPLE OUTPUT:\n");
for (int i = 0; i < CasiTest; i++) printf("%d \n", Risultato[i]);
system("pause");
return 0;
}

```

```

/*
Complessità:  $O(N * M^2 * T)$  dove N = numero di righe; M = numero di colonne; T =
numero di casi di test.
(Nota: la complessità è quadratica rispetto a M per via dell'espansione
iterativa a destra e sinistra per ogni barra dell'istogramma).

Ricorrenza:  $DP[i][j] \Rightarrow \text{vale } DP[i-1][j] + 1$  Se l'elemento nella matrice è 0 e  $i > 0$ 
vale 1 Se l'elemento nella matrice è 0 e  $i == 0$  (Caso base)
vale 0 Se l'elemento nella matrice è 1
*/

```

4. Output Atteso:

```

SAMPLE INPUT:

0 1 1 0 1 1 0
0 0 0 0 0 1 0
1 0 0 0 0 0 1
0 1 0 0 0 0 1
1 1 0 0 0 1 0
1 1 0 1 1 0 0

0 1 1 0 1 1 0
0 0 0 0 0 1 0
1 1 0 0 0 0 1
0 1 0 1 0 0 1
1 1 0 0 0 1 0
1 1 0 1 1 0 0

SAMPLE OUTPUT:
12
6
Premere un tasto per continuare . . .

```

Problema2.

Dato un array `arr[]` di dimensione `n` numeri interi , si scriva un algoritmo per trovare la lunghezza della sottosequenza crescente più lunga, ovvero la sottosequenza più lunga possibile in cui gli elementi sono ordinati in senso crescente. N.B una sottosequenza (diversamente da una sottostringa) può includere elementi non adiacenti.

Questo esercizio viene ricondotto al classico problema della **Longest Increasing Subsequence (LIS)**, che risolvo tramite programmazione dinamica.

1. File di Intestazione: funzioni.h

```
#ifndef FUNZIONI_H
#define FUNZIONI_H
#define MAX 100
/*Definizione VarGlobale*/
extern int DP[MAX];
/*Definizione prototipi*/
void InitDP(int DP[MAX],int n);
int CalcolaDP(int DP[MAX],int arr[MAX],int n);
int max2(int a,int b);
#endif
```

2. File delle Funzioni: funzioni.c

```
#include <stdio.h>
#include <limits.h>
#include <stdlib.h>
#include "funzioni.h" // Include il file header

/*Inizializzazione: Ogni singolo elemento dell'array, preso
singolarmente, costituisce una sottosequenza di lunghezza 1.*/
void InitDP(int DP[MAX],int n){
    for (int i = 0; i < n ; i++)DP[i]=1;
}
/*Ricerca del massimo*/
int max2(int a,int b){return(a>b)?a:b;}

/*Risoluzione del problema:Calcolo la lunghezza della Longest Increasing
Subsequence (LIS)*/
int CalcolaDP(int DP[MAX],int arr[MAX],int n){
    //Passo 0:Definisco la massima lunghezza trovata. Inizializzato a 1 in modo
    tale da gestire il caso base
    int MaXGlobale=1;
```

```

//Passo 1:Inizio con la ricerca
for (int i = 1; i < n; i++){//Per ogni elemento i-esimo di arr[n]
    int valX=arr[i];
    //Vado a valutare tutti gli elementi j precedenti ad i, considero la
//finestra j in [0, i)
    for (int j = 0; j < i; j++){
        //Passo 2: Valuto le condizioni di estensione
        if (arr[j]<valX){//Se il VALORE in j è minore del valore in i,
//allora posso allungare la sequenza in modo crescente
            //La nuova lunghezza in i è il massimo tra quella attuale e
//quella di j incrementata di 1
            DP[i]=max2(DP[i],DP[j]+1);
        }
    }
    //Aggiorno il massimo globale se la sottosequenza ottimale
//terminante in i è la più lunga trovata finora
    MaXGlobale=max2(DP[i],MaXGlobale);
}
//Passo 3:Ritorna la lunghezza della sottosequenza crescente più lunga
return MaXGlobale;
}

```

3. Programma Principale: main.c

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include "funzioni.h" // Include le funzioni definite in funzioni.c
int main() {
    //1.Preparazione dei 3 casi di test
    //Caso1:
    int arr1[] = {10, 9, 2, 5, 3, 7, 101, 18};
    int n1 = 8;
    //Caso2:
    int arr2[] = {50, 40, 30, 20, 10};
    int n2 = 5;
    //Caso3:
    int arr3[] = {2, 4, 6, 8, 10, 12};
    int n3 = 6;
    //Array di puntatori agli array di test
    int* test_arrays[] = {arr1, arr2, arr3};
    //Array contenente le dimensioni corrispondenti
}

```

```

int test_sizes[] = {n1, n2, n3};
int TEST=3;
// Array per salvare i risultati calcolati
int risultati[3];
//Definisco la mia DP
int DP[MAX];
//2.Stampa dell'input
printf("SAMPLE INPUT:\n");
for (int i = 0; i < TEST; i++) {
    // Ciclo interno per stampare gli elementi dell'array corrente
    for(int j = 0; j < test_sizes[i]; j++)printf("%d ", test_arrays[i][j]);
    printf("\n");
}
//3.Calcolo la DP per ogni problema iesimo che abbiamo definito
for (int i = 0; i < TEST; i++) {
    InitDP(DP, test_sizes[i]);
    risultati[i] = CalcolaDP(DP, test_arrays[i], test_sizes[i]);
}
//4.Stampa dei risultati
printf("\n");
printf("SAMPLE OUTPUT:\n");
for (int i = 0; i < TEST; i++) printf("%d\n", risultati[i]);

system("pause");
return 0;
}

```

/*

Complessità: Il metodo CalcolaDP ha una complessità $O(n^2)$,pertanto valutandolo su T casi, avrò $O(T*n^2)$

Ricorrenza:

Caso base: $DP[i]=1$ per ogni i appartenente all'intervallo $[0, n-1]$

Passo Ricorsivo: $DP[i] = \max(DP[j] + 1)$ per ogni j tale che $0 \leq j < i$ e $arr[j] < arr[i]$

*/

4. Output Atteso:

```
SAMPLE INPUT:
10 9 2 5 3 7 101 18
50 40 30 20 10
2 4 6 8 10 12

SAMPLE OUTPUT:
4
1
6
Premere un tasto per continuare . . .
```

Test 1 [10, 9, 2, 5, 3, 7, 101, 18] => Output: 4

La sottosequenza strettamente crescente più lunga è composta da 4 elementi.

Ad esempio: {2, 3, 7, 101} oppure {2, 5, 7, 18}.

Test 2 [50, 40, 30, 20, 10] => Output: 1

L'array è ordinato in senso strettamente decrescente. Qualsiasi elemento scelto singolarmente è la sottosequenza crescente più lunga possibile.

Test 3 [2, 4, 6, 8, 10, 12] => Output: 6

L'array è già completamente ordinato in modo crescente. L'intera sequenza costituisce di fatto la LIS (lunghezza 6).